EVENTFORGE

University Event Planning & Coordination System

COS 214 Practical 4 - TaskForge 2026

System Design, UML Portfolio, Class & Function Implementation Specification

Team size	3 students
Language	C++11
Executable	taskforge
Design patterns	Composite, Iterator, State, Decorator
Team members	Person 1: _____ Person 2: _____ Person 3: _____

Design basis: COS214 Practical 4 (TaskForge)
Function-specification presentation adapted from the detailed style used in COS132 Practical 9.

1. System Concept and Problem

EventForge is a university event-planning system. It coordinates the work needed to organise a real event such as an Open Day, career fair, sports day or society festival. The event is broken into sections, activity groups, task lists and individual tasks. The system is deliberately designed as one believable application rather than four separate design-pattern demonstrations.

1.1 Why this domain fits TaskForge

- University events naturally form a recursive part-whole hierarchy: event plan -> event section -> activity group -> task list -> individual event task.
- Different users need different traversals: an event coordinator may inspect the complete event plan, while a team leader may want only tasks that are not yet complete.
- Event tasks have state-dependent behaviour: a planned task must first be marked ready, a blocked task must be resolved before it can continue, and finished work must be reviewed before it is completed.
- Runtime responsibilities such as organiser approval and urgent/last-minute handling can be added to a task without changing the underlying EventTask class.
- Structural changes are realistic: a last-minute task may be added, a task may move to another task list, or an existing task may be wrapped with extra responsibilities.

1.2 Example operational story

The example scenario is a University Open Day. The EventPlan contains a Logistics section. Logistics contains a Venue Setup activity group. Venue Setup contains a Main Hall Setup task list with tasks such as booking the hall, arranging chairs, setting up the sound system and placing direction signs. A last-minute sound-system task can be made urgent and can require organiser approval at runtime. While two traversals are in progress, a new registration-desk task is added; the old iterators detect the structural change and are safely recreated.

2. Requirement-to-Design Mapping

Requirement	EventForge design response
Recursive hierarchy	EventPlan -> EventSection -> ActivityGroup -> TaskList -> EventTask gives four nested levels below the root/client boundary and contains both groups and individual work items.
Composite	WorkComponent provides the common abstraction; WorkGroup owns child components; EventTask is a leaf; concrete group subclasses give domain meaning.
Iterator	DepthFirstIterator visits the whole hierarchy; IncompleteTaskIterator visits only incomplete executable work. Each object stores its own traversal state.
State	EventTask delegates lifecycle behaviour to TaskState objects: Planned, Ready, InProgress, Blocked, Review and Completed.
Decorator	TaskDecorator shares the ExecutableTask abstraction. ApprovalRequiredDecorator and UrgentTaskDecorator can be added and stacked at runtime.
Runtime modification	Adding, moving, removing or runtime-wrapping a task changes the structure version so existing

Requirement	EventForge design response
	traversals can detect the change.
Traversal modification policy	Structural mutation invalidates existing iterators before they dereference stored traversal pointers. State-only changes do not invalidate them.
Ownership	WorkGroup owns its child objects; the outermost decorator owns the wrapped task chain; EventTask owns its current TaskState; EventPlan owns the shared StructureVersion object.
Virtual destructors	Every polymorphic base class (WorkComponent, ExecutableTask, TaskState, TaskDecorator, WorkIterator) has a virtual destructor.
Docker/GitHub	Repository layout, branch split, Makefile, Dockerfile and debugging workflow are specified later in this document.

3. GoF Participant Mapping

Composite

Component: WorkComponent; Composite: WorkGroup; Concrete Composites: EventPlan, EventSection, ActivityGroup and TaskList; Leaf: EventTask. A decorated EventTask is still used through the same WorkComponent/ExecutableTask abstraction.

Iterator

Iterator: WorkIterator; Concrete Iterators: DepthFirstIterator and IncompleteTaskIterator; Aggregate/factory: WorkGroup; Client: reporting and supervisor workflows in main.cpp.

State

Context: EventTask; State: TaskState; Concrete States: PlannedState, ReadyState, InProgressState, BlockedState, ReviewState, CompletedState.

Decorator

Component: ExecutableTask; Concrete Component: EventTask; Decorator: TaskDecorator; Concrete Decorators: ApprovalRequiredDecorator and UrgentTaskDecorator.

4. Abstract Static View of the Whole System

This first view is deliberately simple. Read it from left to right: the event hierarchy uses Composite, each EventTask uses State, optional task behaviour uses Decorator, and the hierarchy can be traversed using Iterator.

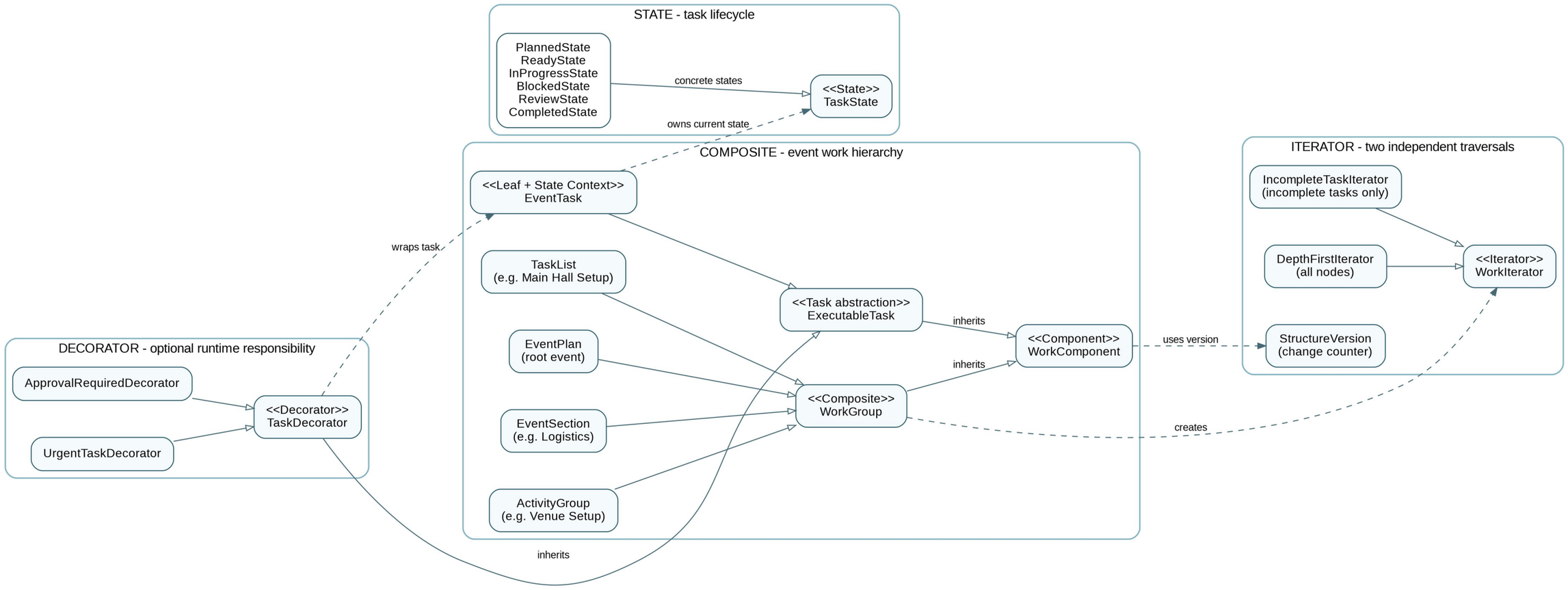

Figure 1. EventForge abstract static architecture. The four required patterns are shown as separate but connected parts of one system.

5. Complete UML Class Diagram

This is the complete class diagram. A larger page is used so class names, relationships and key operations remain readable. The three zoomed views on the following pages explain the most important relationships separately.

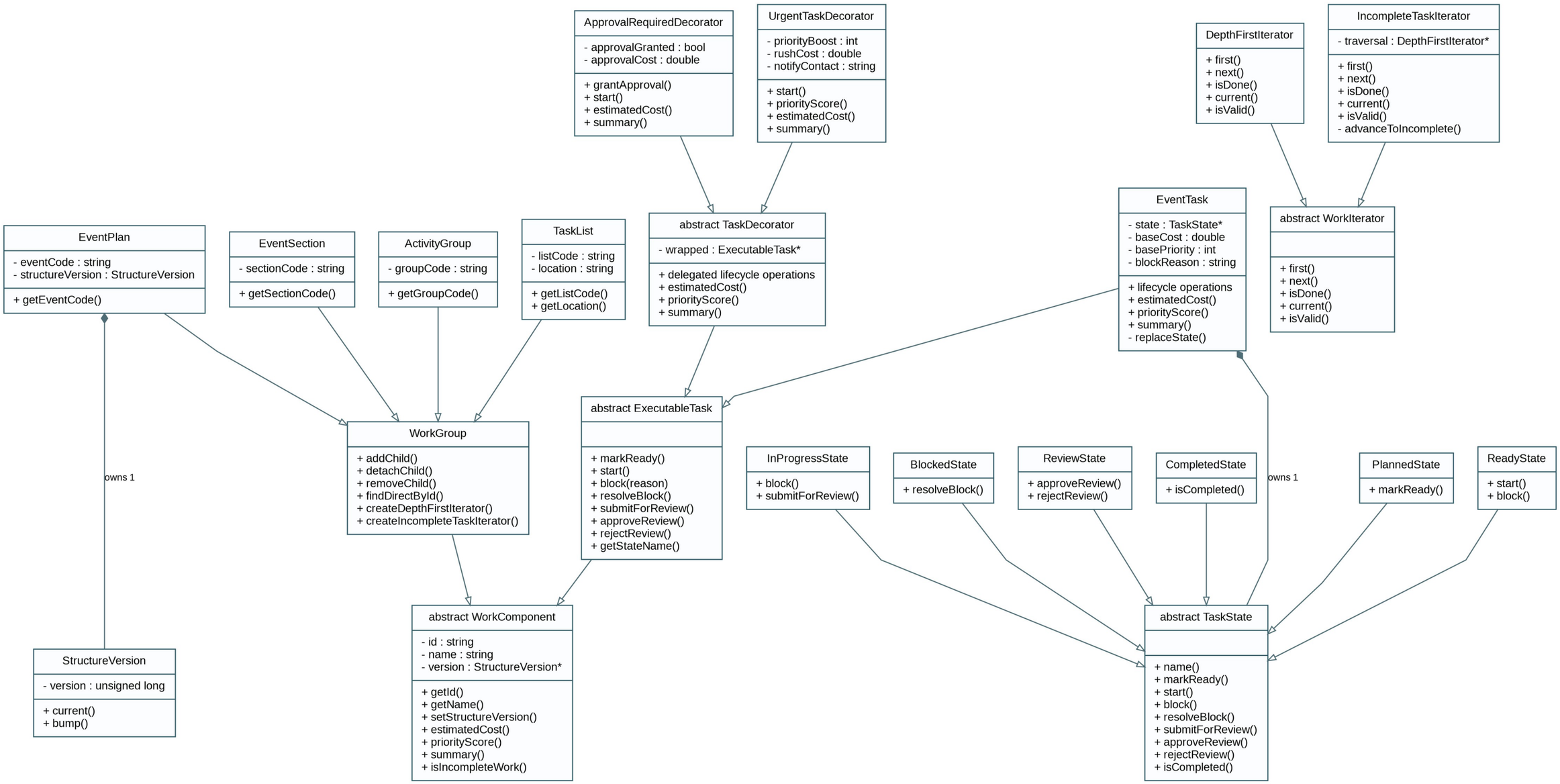

Figure 2. Complete implementation-oriented UML class diagram.

5.1 Composite and Iterator - zoomed view

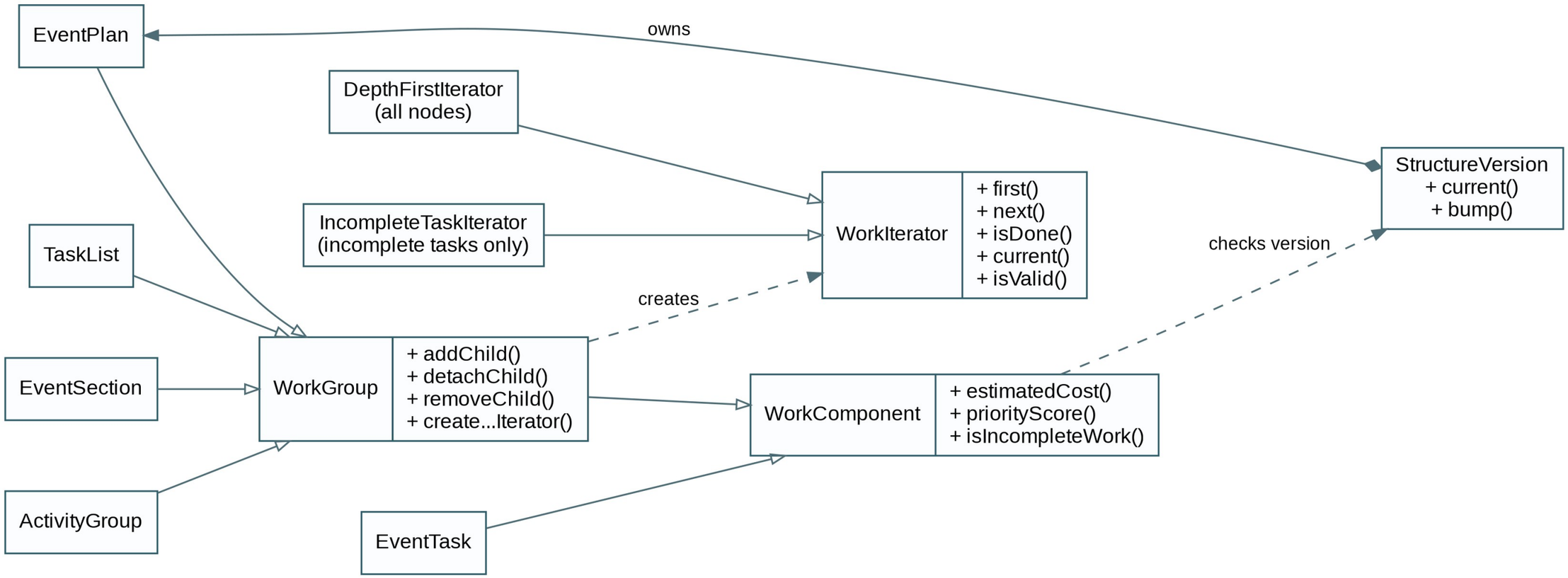

Figure 2A. Zoomed Composite and Iterator relationships.

5.2 State - zoomed view

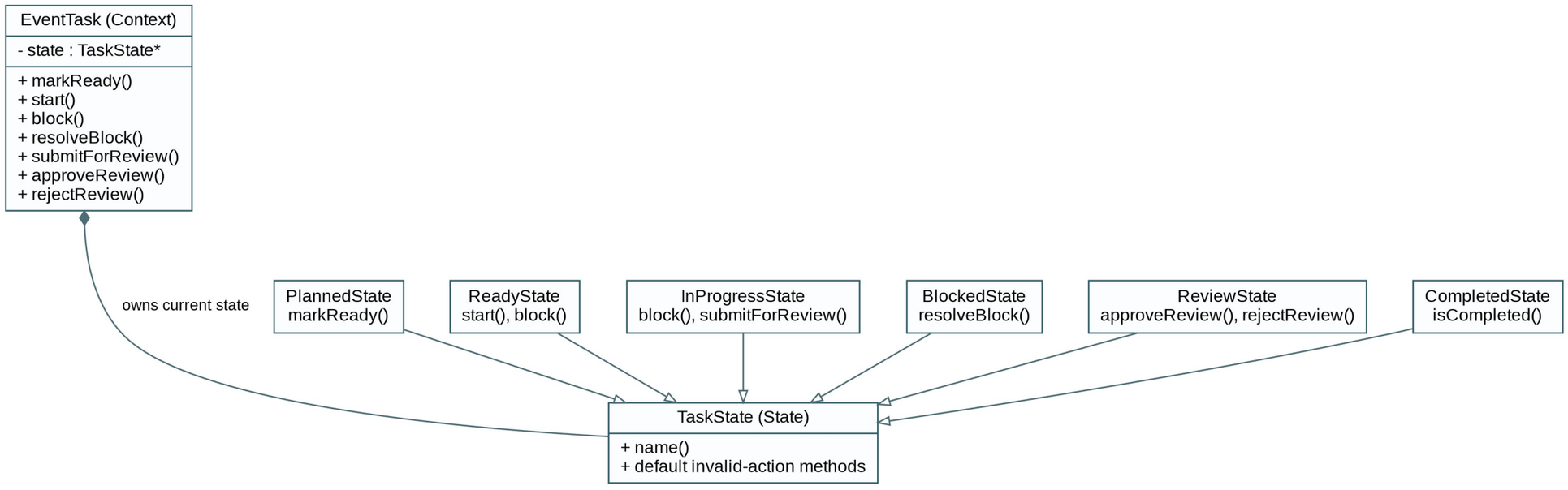

Figure 2B. Zoomed State pattern participants.

5.3 Decorator - zoomed view

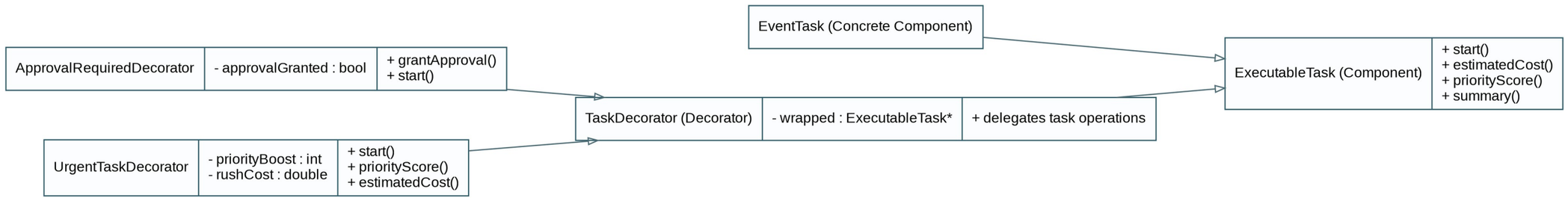

Figure 2C. Zoomed Decorator participants.

6. Ownership, Lifetime and Traversal Policy

6.1 Ownership rules

1. A WorkGroup owns every pointer currently stored in its children vector and deletes those children in its destructor.
2. detachChild transfers ownership to the caller: it removes a child without deleting it. The caller must immediately re-home, wrap, or delete that pointer.
3. removeChild removes and deletes a direct child. It is the safe operation when the object is no longer required.
4. A TaskDecorator owns exactly one wrapped ExecutableTask pointer. Stacked decorators therefore form a single ownership chain.
5. A WorkGroup that stores a decorated task owns only the outermost decorator. Destroying that outermost object recursively destroys every wrapper and eventually the EventTask.
6. EventTask owns exactly one TaskState at a time. A state transition deletes the old state before adopting the newly allocated state.
7. EventPlan owns StructureVersion as a normal member object. Every component and iterator merely keeps a non-owning pointer to that same StructureVersion object.
8. Iterators never own WorkComponent objects. They only retain traversal pointers while the captured structure version remains valid.

6.2 Structural modification policy

EventForge uses a simple version-number policy. EventPlan owns one StructureVersion counter. Whenever the hierarchy changes through addChild, detachChild or removeChild, the counter increases. Each iterator remembers the version it started with. If the live version changes, the iterator marks itself invalid before using stored pointers. The team then creates a fresh iterator. This is easier to understand and safer than trying to repair an iterator after objects have moved or been deleted.

Lifecycle changes such as Ready -> InProgress do not change the hierarchy and therefore do not increment StructureVersion. Existing iterators remain structurally valid and may observe the new state of the same object.

6.3 Why this policy is preferable here

- Trying to keep an iterator live after nested insertions or deletions would require much more complex repair logic.
- Simply storing raw pointers without a version check would be unsafe if a child were removed and deleted.
- A single version counter is easy for all three team members to explain during the tutor demonstration.
- The policy is explicit and traceable from the diagrams to addChild/detachChild/removeChild and iterator isValid().

7. C++ Project Structure

```
taskforge/
|-- include/
|   |-- core/WorkComponent.h
|   |-- core/WorkGroup.h
|   |-- core/StructureVersion.h
|   |-- domain/EventPlan.h
|   |-- domain/EventSection.h
|   |-- domain/ActivityGroup.h
|   |-- domain/TaskList.h
|   |-- task/ExecutableTask.h
|   |-- task/EventTask.h
|   |-- state/TaskState.h
```

```
|   |-- state/PlannedState.h ... CompletedState.h
|   |-- decorator/TaskDecorator.h
|   |-- decorator/ApprovalRequiredDecorator.h
|   |-- decorator/UrgentTaskDecorator.h
|   |-- iterator/WorkIterator.h
|   |-- iterator/DepthFirstIterator.h
|   `-- iterator/IncompleteTaskIterator.h
|-- src/         (matching .cpp files)
|-- docs/        (UML exports + final design PDF)
|-- main.cpp
|-- Makefile
|-- Dockerfile
`-- README.md
```

Keep declarations in headers and behaviour in the corresponding .cpp files. The Makefile must compile all sources using -std=c++11 and produce the executable taskforge.

8. Complete Class and Function Implementation Specification

The following section is written as an implementation contract. Each teammate should implement against these contracts rather than silently changing signatures or semantics on a feature branch. If a contract must change, update the UML/specification first and coordinate the interface change before merging.

8.1 StructureVersion

Responsibility	Stores one simple change counter for the EventPlan hierarchy. Iterators use the counter to detect when add/move/remove operations make an old traversal unsafe.
Parent / base	None
GoF role	Iterator support / modification policy
Owns	No dynamic memory
Header	include/core/StructureVersion.h
Implementation	src/core/StructureVersion.cpp

```
StructureVersion()
```

Purpose. Initialises the hierarchy change counter.

Parameters. None.

Returns. Constructor; no return value.

Implementation requirements.

- Set version to 0.
- Do not allocate memory or access hierarchy objects.

```
unsigned long current() const
```

Purpose. Returns the current structural version.

Parameters. None.

Returns. Current version value.

Implementation requirements.

- Return version without modifying it.

```
void bump()
```

Purpose. Records that the recursive structure has changed.

Parameters. None.

Returns. void

Implementation requirements.

- Increment version exactly once for the structural operation calling bump().
- Do not bump for lifecycle/state changes.

Edge / invalid behaviour. Unsigned overflow is practically irrelevant for the practical; no wraparound handling is required.

8.2 WorkComponent

Responsibility	Defines the common abstraction for both groups and individual/decorated tasks.
Parent / base	None
GoF role	Composite Component
Owns	No child/state ownership; stores only a non-owning StructureVersion pointer
Header	include/core/WorkComponent.h
Implementation	src/core/WorkComponent.cpp

WorkComponent(const std::string& id, const std::string& name)

Purpose. Initialises identity shared by every hierarchy node.

Parameters. id: stable identifier; name: human-readable label.

Returns. Constructor.

Implementation requirements.

- Copy id and name into member variables.
- Initialise the version pointer to NULL until the component is attached to an EventPlan hierarchy.

virtual ~WorkComponent()

Purpose. Provides safe polymorphic destruction.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Do not delete StructureVersion because this class does not own it.

const std::string& getId() const

Purpose. Returns the stable component identifier.

Parameters. None.

Returns. Const reference to id.

Implementation requirements.

- Return the stored id directly.

const std::string& getName() const

Purpose. Returns the display name.

Parameters. None.

Returns. Const reference to name.

Implementation requirements.

- Return the stored name directly.

virtual void setStructureVersion(StructureVersion* version)

Purpose. Attaches the node to the hierarchy StructureVersion object.

Parameters. version: non-owning pointer; may be NULL when detached.

Returns. void

Implementation requirements.

- Store the pointer only.
- WorkGroup overrides this operation to propagate the same StructureVersion object to descendants.

Pattern / ownership notes. The version pointer is never deleted by WorkComponent.

```
StructureVersion* getStructureVersion() const
```

Purpose. Allows iterators/factories to obtain the hierarchy StructureVersion object.

Parameters. None.

Returns. Non-owning pointer.

Implementation requirements.

- Return the stored pointer.

```
virtual double estimatedCost() const = 0
```

Purpose. Provides a uniform cost query across leaves, decorators and groups.

Parameters. None.

Returns. Estimated total cost.

Implementation requirements.

- Pure virtual. WorkGroup sums children; tasks/decorators calculate their own effective cost.

```
virtual int priorityScore() const = 0
```

Purpose. Provides a uniform priority query.

Parameters. None.

Returns. Priority score, designed in the range 0-10.

Implementation requirements.

- Pure virtual. Groups return the maximum child priority; decorators may modify the wrapped value.

```
virtual std::string summary() const = 0
```

Purpose. Returns one concise diagnostic/report line for the node.

Parameters. None.

Returns. Formatted summary string.

Implementation requirements.

- Pure virtual. Do not print directly so callers can choose where output goes.

```
virtual bool isIncompleteWork() const
```

Purpose. Supports the incomplete-task traversal selection rule without client-side type checks.

Parameters. None.

Returns. false for ordinary group nodes; executable tasks override appropriately.

Implementation requirements.

- Base implementation returns false.

```
virtual std::size_t iteratorChildCount() const = 0
```

Purpose. Internal traversal hook used by iterator classes.

Parameters. None.

Returns. Number of children visible to traversal.

Implementation requirements.

- WorkGroup returns children.size(); leaf/task classes return 0.

Pattern / ownership notes. Keep this protected (or private with iterator friendship) so main.cpp does not use it as a substitute for Iterator.

```
virtual WorkComponent* iteratorChildAt(std::size_t index) const = 0
```

Purpose. Internal traversal hook for one child.

Parameters. index: direct-child index.

Returns. Child pointer or NULL when out of range.

Implementation requirements.

- WorkGroup returns the indexed child after range checking; leaf/task classes return NULL.

Pattern / ownership notes. Iterator infrastructure only; not a public client traversal API.

8.3 WorkGroup

Responsibility	Implements the recursive composite storage and shared behaviour for all domain grouping levels.
Parent / base	WorkComponent
GoF role	Composite
Owns	Owns every WorkComponent* currently in children
Header	include/core/WorkGroup.h
Implementation	src/core/WorkGroup.cpp

```
WorkGroup(const std::string& id, const std::string& name)
```

Purpose. Constructs an empty composite group.

Parameters. id, name.

Returns. Constructor.

Implementation requirements.

- Call WorkComponent constructor.
- Initialise children as an empty vector.

```
virtual ~WorkGroup()
```

Purpose. Recursively destroys the owned hierarchy below this group.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Iterate through children and delete each pointer.
- Clear the vector afterwards.
- Do not delete the shared StructureVersion.
- Do not call bump() while the hierarchy is being destroyed.

Pattern / ownership notes. Because WorkComponent has a virtual destructor, deleting a child through WorkComponent* correctly invokes the concrete destructor.

```
bool addChild(WorkComponent* child)
```

Purpose. Adds a direct child and transfers ownership to this group.

Parameters. child: dynamically allocated component pointer.

Returns. true if attached; false if rejected.

Implementation requirements.

- Reject NULL.
- Reject if the same pointer or same direct-child id is already stored.
- Push child into children.
- Call child->setStructureVersion(getStructureVersion()) so the entire attached subtree uses this hierarchy StructureVersion object.
- If a StructureVersion pointer is present, call bump() once.

Edge / invalid behaviour. On failure, ownership remains with the caller.

`WorkComponent* detachChild(const std::string& childId)`

Purpose. Removes a direct child without deleting it so it can be moved or decorated.

Parameters. childId: id of a direct child.

Returns. Detached pointer, or NULL if not found.

Implementation requirements.

- Search only direct children.
- When id matches, erase that vector element but do not delete the object.
- Set the detached subtree StructureVersion pointer to NULL.
- Bump the structure version counter once.
- Return the pointer to the caller.

Pattern / ownership notes. Ownership transfers from the group to the caller.

Edge / invalid behaviour. Caller must re-home, wrap or delete the returned pointer to avoid a leak.

`bool removeChild(const std::string& childId)`

Purpose. Permanently removes and destroys a direct child.

Parameters. childId.

Returns. true if deleted; false if not found.

Implementation requirements.

- Call detachChild(childId).
- If NULL, return false.
- Delete the detached pointer.
- Return true.

Pattern / ownership notes. detachChild performs the single required version bump; removeChild must not bump a second time.

`WorkComponent* findDirectById(const std::string& childId) const`

Purpose. Finds a direct child without exposing the container.

Parameters. childId.

Returns. Non-owning child pointer or NULL.

Implementation requirements.

- Linearly scan children and compare getId().
- Return the matching pointer without transferring ownership.

`void setStructureVersion(StructureVersion* version)`

Purpose. Attaches this group and all descendants to one StructureVersion object.

Parameters. version: non-owning pointer or NULL.

Returns. void

Implementation requirements.

- Call WorkComponent::setStructureVersion(version).
- Iterate through current children and call child->setStructureVersion(version).

Pattern / ownership notes. This propagation keeps an already-built subtree consistent when attached or detached.

`double estimatedCost() const`

Purpose. Computes total estimated cost recursively.

Parameters. None.

Returns. Sum of child costs.

Implementation requirements.

- Initialise total to 0.
- For every child, add child->estimatedCost().
- Return total.

`int priorityScore() const`

Purpose. Represents group urgency as its most urgent descendant.

Parameters. None.

Returns. Maximum child priority, or 0 for an empty group.

Implementation requirements.

- Initialise maxPriority to 0.
- For every child, compare child->priorityScore().
- Return the maximum.

`std::string summary() const`

Purpose. Builds a concise group summary.

Parameters. None.

Returns. String containing group name, child count, estimated cost and max priority.

Implementation requirements.

- Use a string stream.
- Do not recursively print every child; traversal controls reporting order.

`WorkIterator* createDepthFirstIterator()`

Purpose. Creates an independent complete-hierarchy traversal rooted at this group.

Parameters. None.

Returns. New DepthFirstIterator* owned by caller.

Implementation requirements.

- Allocate a DepthFirstIterator using this group as the root and pass the shared StructureVersion pointer.
- The iterator captures the current version independently.

`WorkIterator* createIncompleteTaskIterator()`

Purpose. Creates an independent traversal that yields only incomplete executable work.

Parameters. None.

Returns. New IncompleteTaskIterator* owned by caller.

Implementation requirements.

- Allocate an IncompleteTaskIterator rooted at this group.
- Its internal traversal must have independent state from any other iterator.

`std::size_t iteratorChildCount() const`

Purpose. Traversal-only direct child count.

Parameters. None.

Returns. children.size().

Implementation requirements.

- Return children.size().

`WorkComponent* iteratorChildAt(std::size_t index) const`

Purpose. Traversal-only indexed access.

Parameters. index.

Returns. Pointer or NULL.

Implementation requirements.

- If index >= children.size(), return NULL.
- Otherwise return children[index].

Pattern / ownership notes. Does not transfer ownership.

8.4 EventPlan

Responsibility	Root composite representing one university event. It also owns the shared StructureVersion.
Parent / base	WorkGroup
GoF role	Concrete Composite
Owns	StructureVersion member + inherited children
Header	include/domain/EventPlan.h
Implementation	src/domain/EventPlan.cpp

Domain attributes: eventCode : std::string; structureVersion : StructureVersion.

EventPlan(const std::string& id, const std::string& name, const std::string& eventCode)

Purpose. Creates the root event and connects the inherited hierarchy to its owned version counter.

Parameters. id, name, eventCode.

Returns. Constructor.

Implementation requirements.

- Call WorkGroup constructor.
- Store eventCode.
- After structureVersion exists, call setStructureVersion(&structureVersion).

const std::string& getEventCode() const

Purpose. Returns the event code.

Parameters. None.

Returns. Const reference.

Implementation requirements.

- Return eventCode.

8.5 EventSection

Responsibility	Composite for one major section of the event, for example Logistics, Marketing or Catering.
Parent / base	WorkGroup
GoF role	Concrete Composite
Owns	Inherited child ownership only
Header	include/domain/EventSection.h
Implementation	src/domain/EventSection.cpp

Domain attributes: sectionCode : std::string.

EventSection(const std::string& id, const std::string& name, const std::string& sectionCode)

Purpose. Initialises an event-section composite.

Parameters. id, name, sectionCode.

Returns. Constructor.

Implementation requirements.

- Call WorkGroup constructor.
- Store sectionCode.

const std::string& getSectionCode() const

Purpose. Returns the section code.

Parameters. None.

Returns. Const reference.

Implementation requirements.

- Return sectionCode.

8.6 ActivityGroup

Responsibility	Composite for one group of related activities inside a section, for example Venue Setup or Guest Registration.
Parent / base	WorkGroup
GoF role	Concrete Composite
Owns	Inherited child ownership only
Header	include/domain/ActivityGroup.h
Implementation	src/domain/ActivityGroup.cpp

Domain attributes: groupCode : std::string.

```
ActivityGroup(const std::string& id, const std::string& name, const std::string& groupCode)
```

Purpose. Initialises an activity-group composite.

Parameters. id, name, groupCode.

Returns. Constructor.

Implementation requirements.

- Call WorkGroup constructor.
- Store groupCode.

```
const std::string& getGroupCode() const
```

Purpose. Returns the activity-group code.

Parameters. None.

Returns. Const reference.

Implementation requirements.

- Return groupCode.

8.7 TaskList

Responsibility	Composite for a smaller list of tasks within an activity group, for example Main Hall Setup.
Parent / base	WorkGroup
GoF role	Concrete Composite
Owns	Inherited child ownership only
Header	include/domain/TaskList.h
Implementation	src/domain/TaskList.cpp

Domain attributes: listCode : std::string; location : std::string.

```
TaskList(const std::string& id, const std::string& name, const std::string& listCode, const std::string& location)
```

Purpose. Initialises a task-list composite.

Parameters. id, name, listCode, location.

Returns. Constructor.

Implementation requirements.

- Call WorkGroup constructor.
- Store listCode and location.

```
const std::string& getListCode() const
```

Purpose. Returns the task-list code.

Parameters. None.

Returns. Const reference.

Implementation requirements.

- Return listCode.

```
const std::string& getLocation() const
```

Purpose. Returns the event location associated with this task list.

Parameters. None.

Returns. Const reference.

Implementation requirements.

- Return location.

8.8 ExecutableTask

Responsibility	Defines the common interface that both EventTask and task decorators must support.
Parent / base	WorkComponent
GoF role	Decorator Component + task abstraction
Owns	No additional ownership at this level
Header	include/task/ExecutableTask.h
Implementation	No .cpp required beyond virtual destructor if desired

```
virtual ~ExecutableTask()
```

Purpose. Ensures safe deletion through ExecutableTask*.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool markReady() = 0
```

Purpose. Requests Planned -> Ready.

Parameters. None.

Returns. true on valid transition.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool start() = 0
```

Purpose. Requests Ready -> InProgress; decorators may intercept.

Parameters. None.

Returns. true when execution starts.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool block(const std::string& reason) = 0
```

Purpose. Requests Ready/InProgress -> Blocked.

Parameters. reason.

Returns. true when blocked.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool resolveBlock() = 0
```

Purpose. Requests Blocked -> Ready.

Parameters. None.

Returns. true when resolved.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool submitForReview() = 0
```

Purpose. Requests InProgress -> Review.

Parameters. None.

Returns. true on transition.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool approveReview() = 0
```

Purpose. Requests Review -> Completed.

Parameters. None.

Returns. true on transition.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual bool rejectReview() = 0
```

Purpose. Requests Review -> Ready.

Parameters. None.

Returns. true on transition.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

```
virtual std::string getStateName() const = 0
```

Purpose. Returns lifecycle state name through undecorated/decorated abstractions.

Parameters. None.

Returns. State name string.

Implementation requirements.

- Pure virtual declaration; concrete behaviour is provided by EventTask or delegated by TaskDecorator.

std::size_t iteratorChildCount() const

Purpose. Marks executable tasks as leaves for hierarchy traversal.

Parameters. None.

Returns. 0.

Implementation requirements.

- Return 0.

WorkComponent* iteratorChildAt(std::size_t index) const

Purpose. Leaf traversal hook.

Parameters. index ignored.

Returns. NULL.

Implementation requirements.

- Return NULL for every index.

8.9 EventTask

Responsibility	Concrete executable work item and State Context.
Parent / base	ExecutableTask
GoF role	Composite Leaf + State Context
Owns	Owns current TaskState*
Header	include/task/EventTask.h
Implementation	src/task/EventTask.cpp

EventTask(const std::string& id, const std::string& name, double baseCost, int basePriority)

Purpose. Creates a task in Planned state.

Parameters. id, name, baseCost, basePriority.

Returns. Constructor.

Implementation requirements.

- Call ExecutableTask/WorkComponent constructor.
- Store non-negative baseCost.
- Clamp or validate basePriority to the agreed 0-10 range.
- Set blockReason to empty.
- Allocate state = new PlannedState().

Pattern / ownership notes. EventTask immediately owns the allocated state.

~EventTask()

Purpose. Destroys current state.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Delete state.
- Set state to NULL defensively.

bool markReady()

Purpose. Delegates the markReady lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->markReady(*this).

- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool start()`

Purpose. Delegates the start lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->start(*this).
- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool block(const std::string& reason)`

Purpose. Delegates the block lifecycle request to the current State object.

Parameters. reason: explanation stored when blocking.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->block(*this, reason).
- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool resolveBlock()`

Purpose. Delegates the resolveBlock lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->resolveBlock(*this).
- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool submitForReview()`

Purpose. Delegates the submitForReview lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->submitForReview(*this).
- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool approveReview()`

Purpose. Delegates the approveReview lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->approveReview(*this).

- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`bool rejectReview()`

Purpose. Delegates the rejectReview lifecycle request to the current State object.

Parameters. None.

Returns. true for a valid state-dependent action; false otherwise.

Implementation requirements.

- Delegate to state->rejectReview(*this).
- Return the boolean result from the current state object.
- Do not implement lifecycle decisions with a switch or central if/else chain.

Pattern / ownership notes. State pattern: EventTask is Context; concrete TaskState chooses behaviour.

`std::string getStateName() const`

Purpose. Returns current state name.

Parameters. None.

Returns. String.

Implementation requirements.

- Return state->name().

`double estimatedCost() const`

Purpose. Returns the undecorated task cost.

Parameters. None.

Returns. baseCost.

Implementation requirements.

- Return baseCost.

`int priorityScore() const`

Purpose. Returns the undecorated task priority.

Parameters. None.

Returns. basePriority.

Implementation requirements.

- Return basePriority.

`std::string summary() const`

Purpose. Reports identity, state, cost, priority and block reason if relevant.

Parameters. None.

Returns. Formatted string.

Implementation requirements.

- Build a concise string using getName(), getStateName(), estimatedCost() and priorityScore().
- Append blockReason only when non-empty.

`bool isIncompleteWork() const`

Purpose. Allows IncompleteTaskIterator to select unfinished tasks polymorphically.

Parameters. None.

Returns. true unless current state is Completed.

Implementation requirements.

- Return !state->isCompleted().

void replaceState(TaskState* nextState)

Purpose. Internal transition hook used through TaskState helper logic.

Parameters. nextState: newly allocated state.

Returns. void

Implementation requirements.

- Reject NULL defensively.
- Delete the old state.
- Assign state = nextState.

Pattern / ownership notes. Keep private. TaskState receives controlled friend/helper access; client code must use lifecycle requests instead.

void setBlockReason(const std::string& reason)

Purpose. Internal helper used when entering Blocked state.

Parameters. reason.

Returns. void

Implementation requirements.

- Store reason.

void clearBlockReason()

Purpose. Clears stale reason after resolving a block.

Parameters. None.

Returns. void

Implementation requirements.

- Set blockReason to empty string.

8.10 TaskState

Responsibility	Encapsulates lifecycle-specific behaviour and supplies default rejection for invalid actions.
Parent / base	None
GoF role	State
Owns	No owned dynamic memory
Header	include/state/TaskState.h
Implementation	src/state/TaskState.cpp

virtual ~TaskState()

Purpose. Allows deletion through TaskState*.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- No special cleanup in the base.

virtual std::string name() const = 0

Purpose. Returns concrete state label.

Parameters. None.

Returns. State name.

Implementation requirements.

- Pure virtual; each concrete state returns its exact name.

virtual bool markReady(EventTask& task)

Purpose. Default handler for an invalid markReady request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool start(EventTask& task)
```

Purpose. Default handler for an invalid start request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool block(EventTask& task, const std::string& reason)
```

Purpose. Default handler for an invalid block request in states that do not override it.

Parameters. task context; reason.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool resolveBlock(EventTask& task)
```

Purpose. Default handler for an invalid resolveBlock request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool submitForReview(EventTask& task)
```

Purpose. Default handler for an invalid submitForReview request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool approveReview(EventTask& task)
```

Purpose. Default handler for an invalid approveReview request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".

- Do not change the EventTask state.
- Return false.

```
virtual bool rejectReview(EventTask& task)
```

Purpose. Default handler for an invalid rejectReview request in states that do not override it.

Parameters. task context.

Returns. false.

Implementation requirements.

- Optionally emit a concise diagnostic such as "action not valid while <state>".
- Do not change the EventTask state.
- Return false.

```
virtual bool isCompleted() const
```

Purpose. Lets the context query terminal status polymorphically.

Parameters. None.

Returns. false by default.

Implementation requirements.

- Return false; CompletedState overrides to true.

```
void transition(EventTask& task, TaskState* nextState)
```

Purpose. Protected helper for concrete states to replace the context state safely.

Parameters. task context; nextState allocated by the concrete state.

Returns. void

Implementation requirements.

- Call the EventTask private replaceState(nextState) hook.

Pattern / ownership notes. TaskState is granted controlled friend access; derived states call this inherited protected helper.

8.11 PlannedState

Responsibility	Initial state: task exists but has not yet been marked ready.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/PlannedState.h
Implementation	src/state/PlannedState.cpp

```
std::string name() const
```

Purpose. Returns "Planned".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "Planned".

```
bool markReady(EventTask& task)
```

Purpose. Moves a planned task to Ready.

Parameters. task.

Returns. true.

Implementation requirements.

- Allocate new ReadyState.
- Call transition(task, new ReadyState()).
- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.12 ReadyState

Responsibility	Task is ready and may start or become blocked.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/ReadyState.h
Implementation	src/state/ReadyState.cpp

```
std::string name() const
```

Purpose. Returns "Ready".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "Ready".

```
bool start(EventTask& task)
```

Purpose. Moves Ready -> InProgress.

Parameters. task.

Returns. true.

Implementation requirements.

- Call transition(task, new InProgressState()).
- Return true.

```
bool block(EventTask& task, const std::string& reason)
```

Purpose. Moves Ready -> Blocked and records the reason.

Parameters. task; reason.

Returns. true.

Implementation requirements.

- Call task.setBlockReason(reason).
- Transition to new BlockedState.
- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.13 InProgressState

Responsibility	The event task is currently being carried out.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/InProgressState.h
Implementation	src/state/InProgressState.cpp

std::string name() const

Purpose. Returns "InProgress".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "InProgress".

bool block(EventTask& task, const std::string& reason)

Purpose. Moves InProgress -> Blocked.

Parameters. task; reason.

Returns. true.

Implementation requirements.

- Store reason on task.
- Transition to new BlockedState.
- Return true.

bool submitForReview(EventTask& task)

Purpose. Moves InProgress -> Review after work is finished.

Parameters. task.

Returns. true.

Implementation requirements.

- Transition to new ReviewState.
- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.14 BlockedState

Responsibility	Execution cannot proceed until an obstruction is resolved.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/BlockedState.h
Implementation	src/state/BlockedState.cpp

std::string name() const

Purpose. Returns "Blocked".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "Blocked".

bool resolveBlock(EventTask& task)

Purpose. Clears obstruction and returns task to Ready.

Parameters. task.

Returns. true.

Implementation requirements.

- Call task.clearBlockReason().

- Transition to new ReadyState.
- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.15 ReviewState

Responsibility	Finished event work is waiting for team-leader review.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/ReviewState.h
Implementation	src/state/ReviewState.cpp

`std::string name() const`

Purpose. Returns "Review".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "Review".

`bool approveReview(EventTask& task)`

Purpose. Moves Review -> Completed.

Parameters. task.

Returns. true.

Implementation requirements.

- Transition to new CompletedState.
- Return true.

`bool rejectReview(EventTask& task)`

Purpose. Returns rejected work to Ready so it can be corrected and attempted again.

Parameters. task.

Returns. true.

Implementation requirements.

- Transition to new ReadyState.
- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.16 CompletedState

Responsibility	Terminal state for accepted work.
Parent / base	TaskState
GoF role	Concrete State
Owns	No ownership
Header	include/state/CompletedState.h
Implementation	src/state/CompletedState.cpp

`std::string name() const`

Purpose. Returns "Completed".

Parameters. None.

Returns. String.

Implementation requirements.

- Return "Completed".

```
bool isCompleted() const
```

Purpose. Marks the task terminal.

Parameters. None.

Returns. true.

Implementation requirements.

- Return true.

All lifecycle operations not listed above inherit TaskState's default invalid-action behaviour and return false without changing the task.

8.17 TaskDecorator

Responsibility	Abstract forwarding wrapper that keeps decorated and undecorated tasks usable through ExecutableTask.
Parent / base	ExecutableTask
GoF role	Decorator
Owns	Owns wrapped ExecutableTask*
Header	include/decorator/TaskDecorator.h
Implementation	src/decorator/TaskDecorator.cpp

```
TaskDecorator(ExecutableTask* wrapped)
```

Purpose. Adopts one executable task/wrapper.

Parameters. wrapped: non-NULL pointer whose ownership transfers to this decorator.

Returns. Constructor.

Implementation requirements.

- Store wrapped.
- Reject NULL according to team error policy (assert, exception, or documented invalid object policy).

Pattern / ownership notes. When stacked, each decorator owns the next layer.

```
virtual ~TaskDecorator()
```

Purpose. Destroys the wrapped ownership chain.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Delete wrapped.
- Set wrapped to NULL.

```
bool markReady()
```

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->markReady().

bool start()

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->start().

bool block(const std::string& reason)

Purpose. Default decorator forwarding behaviour.

Parameters. reason.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->block(reason).

bool resolveBlock()

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->resolveBlock().

bool submitForReview()

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->submitForReview().

bool approveReview()

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->approveReview().

bool rejectReview()

Purpose. Default decorator forwarding behaviour.

Parameters. None.

Returns. Boolean result from wrapped task.

Implementation requirements.

- Return wrapped->rejectReview().

std::string getStateName() const

Purpose. Preserves lifecycle query through wrappers.

Parameters. None.

Returns. Wrapped state name.

Implementation requirements.

- Return wrapped->getStateName().

double estimatedCost() const

Purpose. Base decorator forwards effective cost.

Parameters. None.

Returns. Wrapped effective cost.

Implementation requirements.

- Return wrapped->estimatedCost().

```
int priorityScore() const
```

Purpose. Base decorator forwards effective priority.

Parameters. None.

Returns. Wrapped effective priority.

Implementation requirements.

- Return wrapped->priorityScore().

```
std::string summary() const
```

Purpose. Base decorator forwards summary.

Parameters. None.

Returns. Wrapped summary.

Implementation requirements.

- Return wrapped->summary().

```
bool isIncompleteWork() const
```

Purpose. Preserves incomplete-task selection through wrappers.

Parameters. None.

Returns. Wrapped result.

Implementation requirements.

- Return wrapped->isIncompleteWork().

```
void setStructureVersion(StructureVersion* version)
```

Purpose. Ensures the decorated chain shares the hierarchy StructureVersion object.

Parameters. version pointer.

Returns. void

Implementation requirements.

- Call WorkComponent::setStructureVersion(version) for the decorator object.
- Call wrapped->setStructureVersion(version) to propagate to the underlying task/wrappers.

Pattern / ownership notes. This is essential when a detached task is wrapped and then re-attached.

8.18 ApprovalRequiredDecorator

Responsibility	Adds a runtime organiser-approval gate to a task and optionally adds the cost of obtaining that approval.
Parent / base	TaskDecorator
GoF role	Concrete Decorator
Owns	Inherited ownership of wrapped task
Header	include/decorator/ApprovalRequiredDecorator.h
Implementation	src/decorator/ApprovalRequiredDecorator.cpp

```
ApprovalRequiredDecorator(ExecutableTask* wrapped, double approvalCost)
```

Purpose. Wraps a task that may not start until organiser approval has been granted.

Parameters. wrapped; approvalCost.

Returns. Constructor.

Implementation requirements.

- Call TaskDecorator(wrapped).
- Set approvalGranted=false.
- Store non-negative approvalCost.

void grantApproval()

Purpose. Records the runtime organiser approval.

Parameters. None.

Returns. void

Implementation requirements.

- Set approvalGranted=true.

bool start()

Purpose. Adds a precondition before normal task start behaviour.

Parameters. None.

Returns. true only if approval has been granted and the wrapped task can start.

Implementation requirements.

- If approvalGranted is false, return false without calling wrapped->start().
- Otherwise return TaskDecorator::start().

Pattern / ownership notes. Place this decorator outermost when stacked so the approval gate runs before the urgent decorator sends its start notification.

double estimatedCost() const

Purpose. Adds approval processing/controls cost.

Parameters. None.

Returns. Wrapped cost + approvalCost.

Implementation requirements.

- Return wrapped->estimatedCost() + approvalCost.

std::string summary() const

Purpose. Adds approval status to normal task reporting.

Parameters. None.

Returns. Augmented string.

Implementation requirements.

- Take wrapped->summary().
- Append organiser approval status and approval cost.

8.19 UrgentTaskDecorator

Responsibility	Adds urgent/last-minute handling to a task at runtime.
Parent / base	TaskDecorator
GoF role	Concrete Decorator
Owns	Inherited ownership of wrapped task
Header	include/decorator/UrgentTaskDecorator.h
Implementation	src/decorator/UrgentTaskDecorator.cpp

UrgentTaskDecorator(ExecutableTask* wrapped, int priorityBoost, double rushCost, const std::string& notifyContact)

Purpose. Creates an urgent-task wrapper.

Parameters. wrapped task, priority boost, rush cost and notification contact.

Returns. Constructor.

Implementation requirements.

- Call TaskDecorator(wrapped).
- Store a non-negative boost and rush cost.
- Store notifyContact.

`bool start()`

Purpose. Adds urgent notification behaviour before delegating to the wrapped task.

Parameters. None.

Returns. Wrapped start result.

Implementation requirements.

- Record or print a concise urgent-task notification to notifyContact.
- Delegate to TaskDecorator::start().
- Return the delegated result.

Pattern / ownership notes. When ApprovalRequiredDecorator is the outer wrapper, this method runs only after organiser approval.

`int priorityScore() const`

Purpose. Raises the effective priority dynamically.

Parameters. None.

Returns. min(10, wrapped priority + boost).

Implementation requirements.

- Calculate wrapped->priorityScore()+priorityBoost.
- Cap the value at 10.

`double estimatedCost() const`

Purpose. Adds urgent rush cost.

Parameters. None.

Returns. Wrapped cost + rushCost.

Implementation requirements.

- Return wrapped->estimatedCost()+rushCost.

`std::string summary() const`

Purpose. Adds urgent-task notification/priority information.

Parameters. None.

Returns. Augmented summary.

Implementation requirements.

- Append an urgent-task label, notification contact and effective priority to wrapped summary.

8.20 WorkIterator

Responsibility	Common traversal interface.
Parent / base	None
GoF role	Iterator
Owns	No hierarchy ownership
Header	include/iterator/WorkIterator.h
Implementation	No substantial .cpp required

`virtual ~WorkIterator()`

Purpose. Safe polymorphic deletion.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Pure virtual interface contract.

`virtual void first() = 0`

Purpose. Resets traversal to its first eligible element.

Parameters. None.

Returns. void

Implementation requirements.

- Pure virtual interface contract.

`virtual void next() = 0`

Purpose. Advances to the next eligible element.

Parameters. None.

Returns. void

Implementation requirements.

- Pure virtual interface contract.

`virtual bool isDone() const = 0`

Purpose. Reports end of traversal.

Parameters. None.

Returns. Boolean

Implementation requirements.

- Pure virtual interface contract.

`virtual WorkComponent* current() const = 0`

Purpose. Returns current element without transferring ownership.

Parameters. None.

Returns. Non-owning pointer or NULL

Implementation requirements.

- Pure virtual interface contract.

`virtual bool isValid() const = 0`

Purpose. Reports whether structural version still matches.

Parameters. None.

Returns. Boolean

Implementation requirements.

- Pure virtual interface contract.

8.21 DepthFirstIterator

Responsibility	Traverses every node in pre-order without exposing group containers to client code.
Parent / base	WorkIterator
GoF role	Concrete Iterator
Owns	Owns only its traversal stack; never owns hierarchy

	nodes or version pointer
Header	include/iterator/DepthFirstIterator.h
Implementation	src/iterator/DepthFirstIterator.cpp

DepthFirstIterator(WorkComponent* root, StructureVersion* version)

Purpose. Creates an independent DFS traversal.

Parameters. root: non-owning start node; version: non-owning StructureVersion.

Returns. Constructor.

Implementation requirements.

- Store root and version pointer.
- Capture expectedVersion = version ? version->current() : 0.
- Initialise invalidated=false and currentNode=NULL.
- Do not traverse until first() is called.

~DepthFirstIterator()

Purpose. Releases iterator-owned traversal state only.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Clear stack.
- Never delete root, currentNode or StructureVersion pointer.

bool checkVersion()

Purpose. Internal guard that prevents use of stale pointers.

Parameters. None.

Returns. true if usable.

Implementation requirements.

- If already invalidated return false.
- If version pointer exists and version->current()!=expectedVersion, set invalidated=true, set currentNode=NULL, clear stack without dereferencing nodes, return false.
- Otherwise return true.

void first()

Purpose. Initialises pre-order traversal at root.

Parameters. None.

Returns. void

Implementation requirements.

- Clear stack and currentNode.
- Refresh/retain expectedVersion according to chosen reset contract; recommended: capture current structure version so first() on a deliberately reused iterator starts a fresh traversal.
- Set invalidated=false.
- If root is NULL, leave done.
- Set currentNode=root.
- Prepare stack frame(s) so next() can descend to the first child.

void next()

Purpose. Advances DFS pre-order.

Parameters. None.

Returns. void

Implementation requirements.

- Call checkVersion() before dereferencing any stored node.
- If current node has an unvisited child, descend to that child and push/update a frame.
- Otherwise repeatedly pop/advance ancestor frames until an unvisited sibling exists.
- When no frames remain, set currentNode=NULL.

bool isDone() const

Purpose. Indicates no current element or invalidation.

Parameters. None.

Returns. true when traversal ended/invalid.

Implementation requirements.

- If invalidated return true.
- Otherwise return currentNode==NULL.

WorkComponent* current() const

Purpose. Returns current element safely.

Parameters. None.

Returns. Non-owning pointer or NULL.

Implementation requirements.

- Perform version validation before exposing pointer (use a mutable/internal guard design if necessary).
- Return NULL when invalid/done; otherwise currentNode.

bool isValid() const

Purpose. Checks structural validity.

Parameters. None.

Returns. Boolean.

Implementation requirements.

- Compare captured/current structure version without dereferencing hierarchy nodes.
- Return false after mismatch.

8.22 IncompleteTaskIterator

Responsibility	Traverses the same hierarchy but yields only incomplete executable work.
Parent / base	WorkIterator
GoF role	Concrete Iterator / filtered traversal
Owns	Owns an internal DepthFirstIterator*; does not own hierarchy nodes
Header	include/iterator/IncompleteTaskIterator.h
Implementation	src/iterator/IncompleteTaskIterator.cpp

IncompleteTaskIterator(WorkComponent* root, StructureVersion* version)

Purpose. Creates an independent filtered traversal.

Parameters. root and version as non-owning pointers.

Returns. Constructor.

Implementation requirements.

- Allocate its own DepthFirstIterator(root, version).
- Initialise current incomplete node to NULL.

Pattern / ownership notes. Using a private iterator object preserves independent traversal state.

~IncompleteTaskIterator()

Purpose. Destroys only the internal iterator.

Parameters. None.

Returns. Destructor.

Implementation requirements.

- Delete traversal.
- Do not delete hierarchy nodes.

`void advanceToIncomplete()`

Purpose. Internal filter loop.

Parameters. None.

Returns. void

Implementation requirements.

- While internal traversal is valid and not done, inspect traversal->current()->isIncompleteWork().
- If true, stop; otherwise call traversal->next().
- Do not use dynamic_cast or a type-name switch.

`void first()`

Purpose. Resets and finds first incomplete task.

Parameters. None.

Returns. void

Implementation requirements.

- Call traversal->first().
- Call advanceToIncomplete().

`void next()`

Purpose. Moves to next incomplete task.

Parameters. None.

Returns. void

Implementation requirements.

- If traversal invalid/done, do nothing.
- Call traversal->next().
- Call advanceToIncomplete().

`bool isDone() const`

Purpose. Reports no eligible current task or invalidation.

Parameters. None.

Returns. Boolean.

Implementation requirements.

- Return traversal->isDone().

`WorkComponent* current() const`

Purpose. Returns current incomplete task as WorkComponent*.

Parameters. None.

Returns. Non-owning pointer or NULL.

Implementation requirements.

- Return traversal->current().

`bool isValid() const`

Purpose. Delegates structural validity.

Parameters. None.

Returns. Boolean.

Implementation requirements.

- Return traversal->isValid().

9. Important Interaction Recipes

9.1 Building the required four-level hierarchy

EventPlan -> EventSection -> ActivityGroup -> TaskList -> EventTask

The running program must actually create this depth. A simple example is Open Day -> Logistics -> Venue Setup -> Main Hall Setup -> Arrange Chairs.

9.2 Adding decorators to an existing task at runtime

9. Keep a non-owning ExecutableTask* variable to the task when it is first created and attached.
10. Call parentList->detachChild(taskId). The task is removed without deletion, so the caller temporarily owns it.
11. If the task is suddenly urgent, create new UrgentTaskDecorator(task, priorityBoost, rushCost, notifyContact).
12. If organiser approval is also required, wrap the urgent object with ApprovalRequiredDecorator. The approval wrapper becomes the outermost object.
13. Call parentList->addChild(outermost). Ownership moves back to the composite.
14. Use the outermost ExecutableTask pointer from then on. The original EventTask now lives inside the decorator ownership chain.
15. Any iterators created before detach/add are invalid because StructureVersion changed.

9.3 Moving a task between task lists

16. Call sourceList->detachChild(taskId).
17. Check that the returned pointer is not NULL.
18. Call destinationList->addChild(returnedPointer).
19. If adding to the destination fails, re-add the task to the source or delete it; never lose the transferred pointer.
20. Existing iterators are now invalid, so create fresh traversal objects.

10. Runtime Demonstration Scenarios

Scenario A - University Open Day setup

21. Create EventPlan Open Day 2026 with Logistics -> Venue Setup -> Main Hall Setup and at least four EventTasks.
22. Create DepthFirstIterator A and IncompleteTaskIterator B at the same time. Advance them in an interleaved order to prove that each iterator stores its own traversal state.
23. Call start() on a Planned task and show that it fails. Then call markReady() followed by start() and show the valid transition to InProgress.
24. Block Arrange Chairs because the chairs have not arrived. Attempt submitForReview() while blocked (invalid), then resolveBlock(), start again and continue.
25. Detach Setup Sound and add UrgentTaskDecorator plus ApprovalRequiredDecorator at runtime. Starting before grantApproval() must fail; after approval it must succeed.
26. Add a new Registration Desk task while an iterator exists. The old iterator detects the StructureVersion mismatch; create a fresh iterator and show the new task.
27. Move a completed task through Review -> Completed and show that IncompleteTaskIterator no longer returns it.

Scenario B - Last-minute room change

28. Create a second TaskList called Auditorium Setup under the same Venue Setup ActivityGroup.

29. Move Place Direction Signs from Main Hall Setup to Auditorium Setup using detachChild() and addChild().
30. Create fresh iterators and show that depth-first traversal now finds the task under the new list.
31. Add UrgentTaskDecorator to the moved task because the room change is last minute.
32. Show that recursive cost and priority values at TaskList, ActivityGroup and EventPlan levels change automatically.

11. UML Object Diagram

This object snapshot is intentionally specific enough to show composite ownership, decorator stacking and an actual task state at runtime.

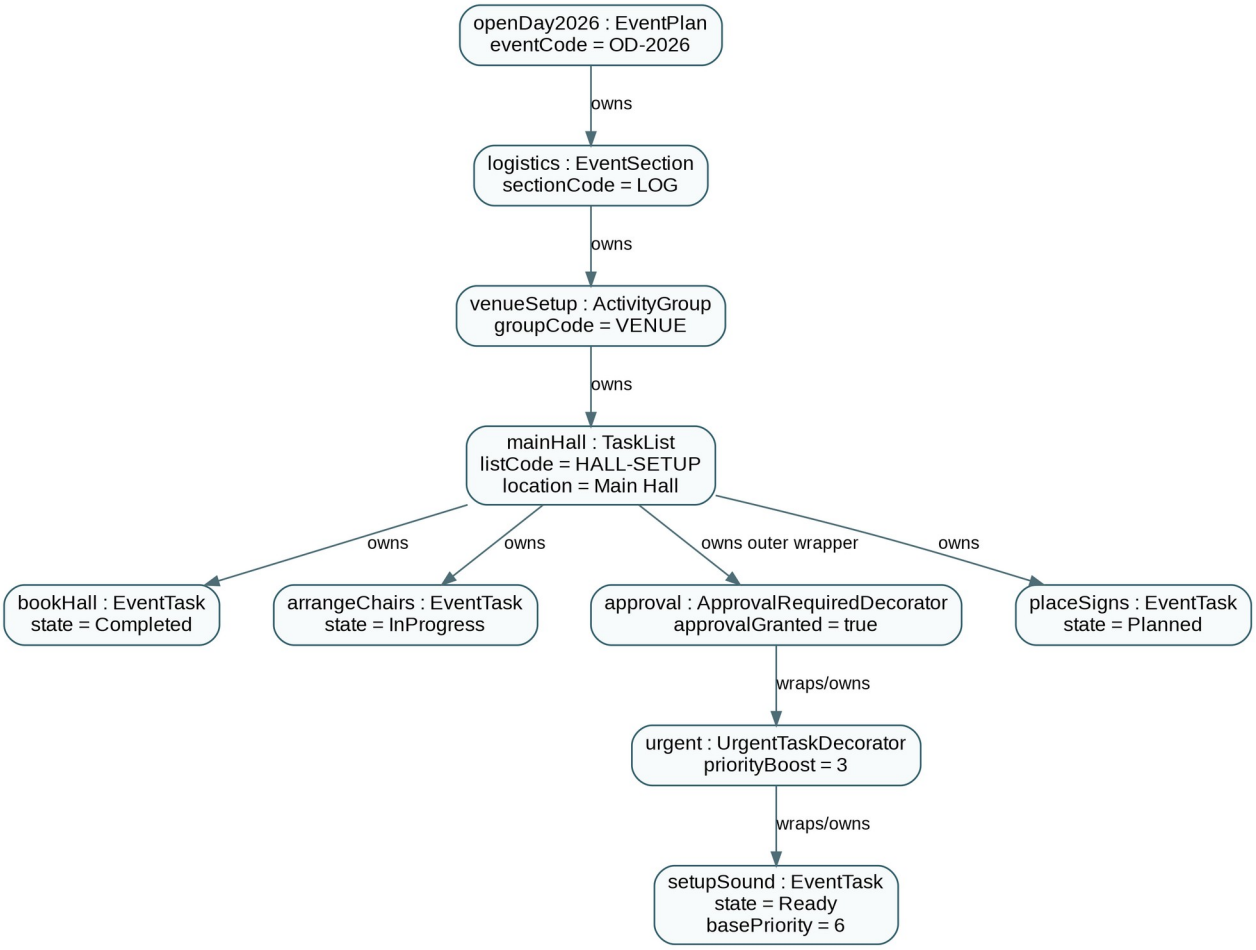

Figure 3. Example runtime object diagram for University Open Day 2026.

12. UML State Diagram

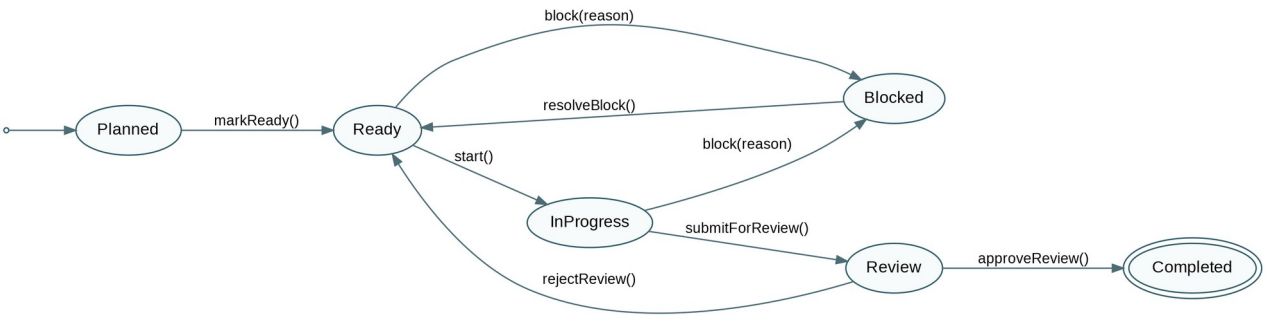

Figure 4. EventTask lifecycle. Actions that are not allowed in the current state return false and do not change the task.

The key design point is that lifecycle decisions reside in state classes, not in a central status switch inside EventTask.

13. UML Activity Diagram Portfolio

13.1 Event setup, runtime configuration and task completion

This workflow uses decisions, a loop after rejected review, a fork/join and a called activity. Responsibility is shown directly in each action label: Event Coordinator, Team Leader and Student Team. If your tutor expects formal swimlane columns, redraw these same actions in three swimlanes without changing the logic.

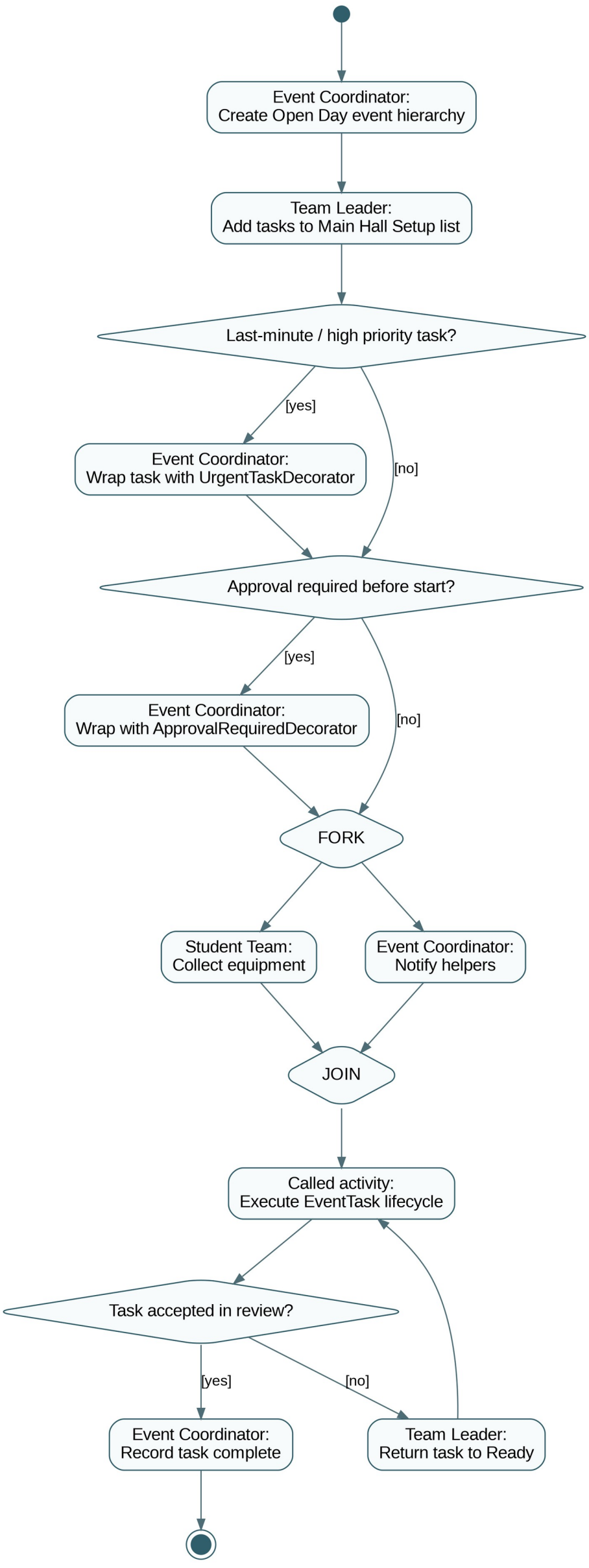

Figure 5. Activity workflow 1 - Open Day task setup and completion.

13.2 Visible hierarchy traversal and structural invalidation

This diagram deliberately exposes iterator first/current/next/isDone operations as part of the larger reporting workflow, satisfying the requirement that at least one Activity Diagram visibly shows traversal rather than hiding it behind a single action.

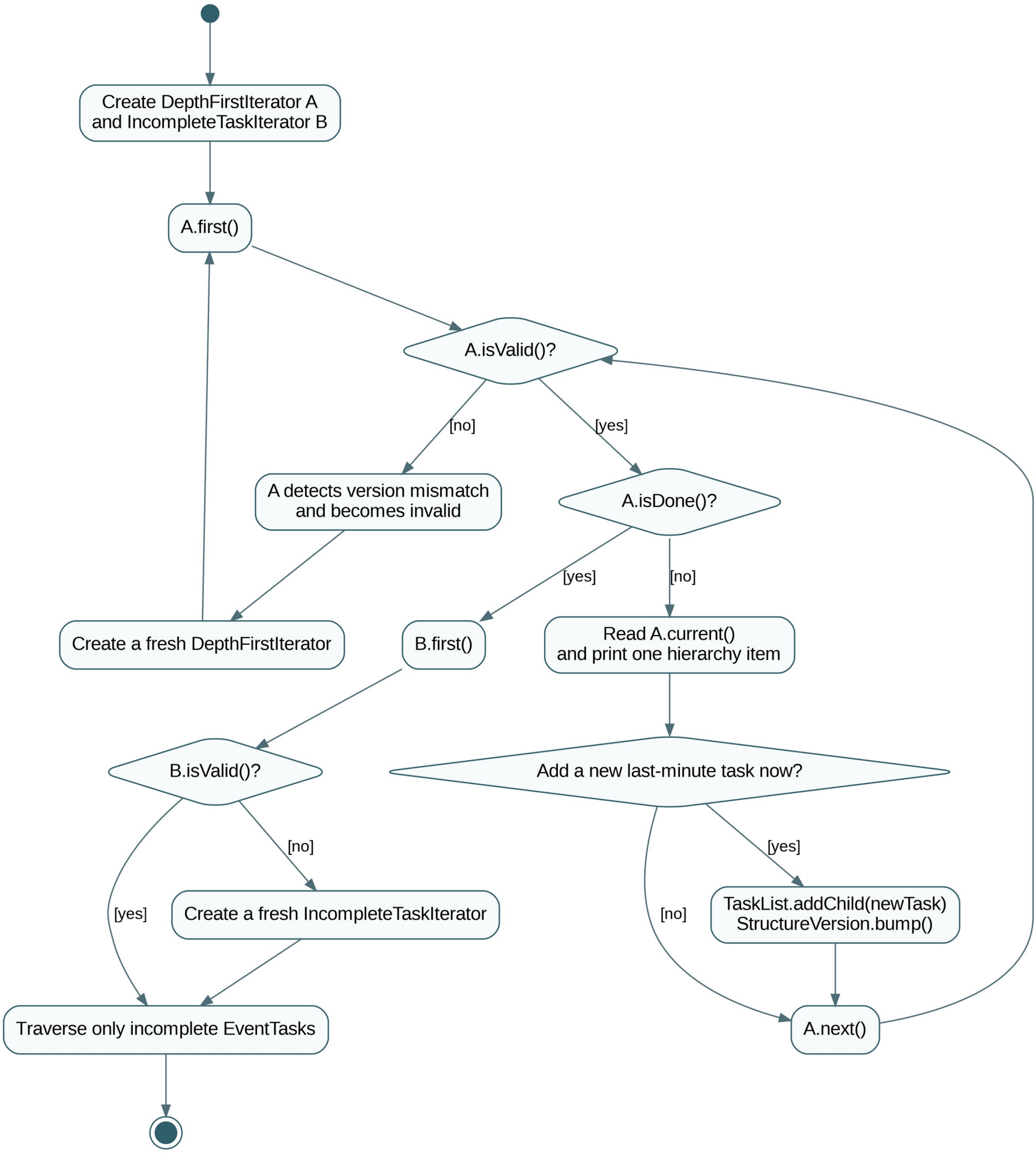

Figure 6. Activity workflow 2 - visible traversal and structural-change policy.

13.3 Runtime decoration plus lifecycle behaviour

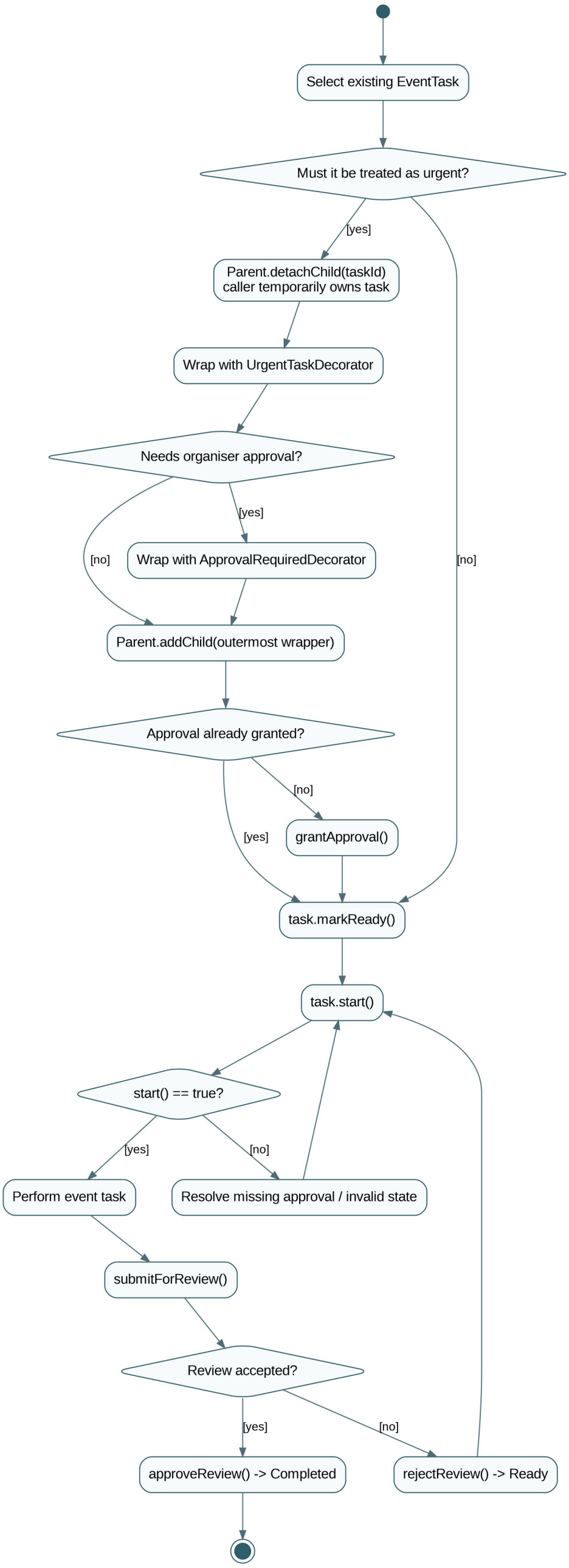

Figure 7. Activity workflow 3 - runtime decoration and EventTask lifecycle.

14. Three-Person Implementation Delegation

The work is split into three clearly named tasks. The team should first agree on the public header signatures from Section 8 and commit those interfaces to main. Each person then works mainly on a separate feature branch so Git history shows genuine individual contributions and merge conflicts are limited.

Task 1 - Person 1: Composite Hierarchy and Ownership

Branch	feature/task1-composite
Main classes	StructureVersion, WorkComponent, WorkGroup, EventPlan, EventSection, ActivityGroup, TaskList
Main responsibility	Build the recursive event hierarchy, child ownership, add/detach/remove behaviour, recursive cost/priority aggregation and structure-version updates.
Tests to own	Nested hierarchy creation, add NULL/duplicate protection, detach without delete, remove with delete, recursive totals, root destruction.
Integration boundary	Publish stable WorkComponent and WorkGroup headers early because Persons 2 and 3 depend on them.

Task 2 - Person 2: EventTask, State and Decorators

Branch	feature/task2-task-state-decorator
Main classes	ExecutableTask, EventTask, TaskState, PlannedState, ReadyState, InProgressState, BlockedState, ReviewState, CompletedState, TaskDecorator, ApprovalRequiredDecorator, UrgentTaskDecorator
Main responsibility	Implement the task lifecycle with valid/invalid state transitions and implement the two stackable runtime decorators.
Tests to own	All valid transitions, invalid start/review actions, block/resolve path, approval gate, urgent priority/cost, stacked decorator ownership.
Integration boundary	Use the agreed WorkComponent interface. Do not change WorkGroup storage or Iterator internals on this branch.

Task 3 - Person 3: Iterators, Integration and Build Tooling

Branch	feature/task3-iterator-integration
Main classes/files	WorkIterator, DepthFirstIterator, IncompleteTaskIterator, main.cpp, Makefile, Dockerfile, README.md
Main responsibility	Implement two independent traversals, StructureVersion invalidation checks, integrated runtime scenarios and the reproducible Docker/build workflow.
Tests to own	Full DFS order, incomplete-only filtering, two simultaneous iterators, invalidation after add/move/remove, final integration scenario, Docker build/run.
Integration boundary	Consume the stable hierarchy/task interfaces. Person 3 coordinates the final main.cpp story after Tasks 1 and 2 merge.

14.1 Shared responsibility - all three people must understand

- The complete UML class diagram and the ownership chain.
- Why WorkGroup is the Composite and EventTask is a leaf.
- How DepthFirstIterator and IncompleteTaskIterator keep independent traversal state.
- Why a structural change invalidates an iterator but a state-only change does not.
- Every EventTask state transition and at least one invalid transition.
- How ApprovalRequiredDecorator and UrgentTaskDecorator can be stacked and who owns the wrapped task.
- How to build/run in Docker and how to run GDB and Valgrind.

15. GitHub Workflow and Merge Plan

15.1 Repository setup

33. Create the GitHub repository immediately; do not wait until the code is finished.
34. Initial main commit: README skeleton, folders, .gitignore, empty Makefile/Dockerfile placeholders and the approved header/interface contracts.
35. Create three feature branches from the same main commit using the branch names in Section 14.
36. Each person commits meaningful increments rather than one final dump.
37. Open pull requests; another person reviews each PR before merge.
38. After each merge, all three people pull main and run make plus the demonstration before continuing.

15.2 Suggested commit sequence

Contributor	Example meaningful commit message
Person 1	Task 1: define StructureVersion and WorkComponent contract
Person 1	Task 1: implement composite ownership and recursive aggregation
Person 1	Task 1: add EventPlan hierarchy classes and tests
Person 2	Task 2: implement EventTask and TaskState default behaviour
Person 2	Task 2: add Planned/Ready/InProgress/Blocked transitions

Person 2	Task 2: add Review/Completed states and decorator tests
Person 3	Task 3: implement depth-first iterator with version invalidation
Person 3	Task 3: add incomplete-task filtered iterator
Person 3	Task 3: add Docker Makefile README and integrated scenarios
All	Review UML against implementation and correct mismatches
All	Add real GDB/Valgrind evidence and final documentation

15.3 Merge order

39. Merge Task 1 / Person 1 first once the hierarchy interfaces and ownership tests are stable.
40. Merge Task 2 / Person 2 next; resolve only interface-level conflicts and preserve the tested State/Decorator behaviour.
41. Person 3 updates Task 3 from main, completes iterator and application integration, and opens the final subsystem PR.
42. Run an integration pass on main. Any bug fix should be a new small branch/PR so Git history shows genuine collaboration.

16. main.cpp Demonstration Blueprint

Do not make a menu that runs one pattern at a time. main.cpp should tell one connected operational story. A recommended five-minute sequence follows.

43. Create the Open Day hierarchy and print a short event heading.
44. Run two independent iterators interleaved.
45. Show an invalid state transition and then the valid lifecycle path.
46. Detach and dynamically decorate Setup Sound; show that start fails before organiser approval and succeeds after grantApproval().
47. Display cost/priority before and after decoration to prove wrappers participate in normal system behaviour.
48. Modify the hierarchy while an iterator exists; show isValid()==false and recreate traversal.
49. Complete one EventTask and show that IncompleteTaskIterator stops returning it.
50. Delete only the EventPlan root and iterator objects at the end. Valgrind should confirm the ownership policy cleans the hierarchy.

17. Testing Checklist

Area	Minimum tests
Composite	Empty group cost/priority; nested recursive totals; add rejects NULL/duplicate; detach transfers ownership; remove deletes; root deletion cleans all descendants.
State	Planned start fails; markReady succeeds; Ready start succeeds; InProgress submitForReview succeeds; Review approveReview succeeds; Completed actions fail; Ready/InProgress block succeeds; Blocked resolve returns Ready.
Decorator	Undecorated task works normally; ApprovalRequiredDecorator blocks start before approval; UrgentTaskDecorator raises priority/cost;

	stacked wrappers preserve lifecycle operations; deleting the outer wrapper deletes the full chain.
Iterator	DFS visits root and every nested object in expected pre-order; IncompleteTaskIterator skips groups and completed tasks; two simultaneous iterators do not share cursor state; structural mutation invalidates existing iterator before pointer dereference.
Integration	Move a task between task lists; decorate after detach; reattach version propagation; recursive group cost/priority changes; a fresh iterator sees the new structure.
Memory	No double-delete after detach/wrap/add; no forgotten detached pointer; iterators do not delete hierarchy nodes; EventTask deletes exactly one state at destruction.

18. GDB and Valgrind Engineering Evidence Plan

Important: this section is a preparation template, not fabricated evidence. The submitted PDF must be updated with a bug your team genuinely encounters and with actual screenshots/output from your final executable.

18.1 GDB evidence to capture

- Compile with -g in the Makefile.
- Set a breakpoint in WorkGroup::detachChild, EventTask::replaceState, or DepthFirstIterator::next.
- Step through a meaningful scenario and inspect children size, current state pointer/name, expectedVersion and live StructureVersion version.
- Capture the actual symptom, root cause, commands/variables inspected and the code correction.
- After correction, rerun the same scenario to demonstrate the fault is gone.

```
gdb ./taskforge
(gdb) break WorkGroup::detachChild
(gdb) run
(gdb) next
(gdb) print children.size()
(gdb) print childId
```

18.2 Valgrind evidence to capture

```
valgrind --leak-check=full --show-leak-kinds=all ./taskforge
```

Final evidence should show no definitely-lost memory originating from your code. If reachable library allocations appear, distinguish them from project-owned leaks rather than claiming a clean result without checking the trace.

18.3 Likely high-value places to investigate if a bug appears

- Decorator ownership after detach/add (double delete or leak).
- State replacement ordering (deleting current state before a concrete state method finishes).
- Iterator using a stored WorkComponent* after a structural version mismatch.
- Moving a task and forgetting to reattach the destination hierarchy StructureVersion object.
- Calling removeChild on a pointer that has already been transferred into a decorator.

19. Docker / Build Specification

19.1 Makefile expectations

- Compile with g++ -std=c++11 -Wall -Wextra -g (add -Werror only if your team can keep the build warning-free).
- Compile all src/*.cpp plus main.cpp with include paths pointing at include/.
- Final target must be named taskforge.
- Provide at least make, make clean and optionally make valgrind targets.

19.2 Dockerfile expectations

- Use a Linux base image available to the team/marker.
- Install g++, make, gdb and valgrind.
- Copy the project into the image or mount it as described in README.
- Set a working directory and demonstrate commands for make, ./taskforge, gdb and valgrind.

20. Submission PDF Assembly Checklist

☐ System description and final team details.

☐ Complete class diagram matching the actual submitted headers.

☐ GoF participant mapping.

☐ Design/ownership rationale and traversal invalidation policy.

☐ Object diagram based on the actual runtime scenario.

☐ EventTask state diagram.

☐ Three substantial activity diagrams, redrawn with explicit UML swimlanes where indicated.

☐ Real GDB bug investigation evidence.

☐ Real final Valgrind evidence.

☐ GitHub repository link and short workflow reflection citing real commits/PRs/issues.

☐ Short contribution statement for all three students.

☐ Cross-check that every important activity-diagram action maps to an actual method in Section 8 / implementation.

☐ Run make in the supplied Docker environment and verify executable name taskforge.

21. Final Design Rationale

EventForge is intentionally simple to explain. Composite models the event as sections, activity groups, task lists and EventTasks. Iterator provides a full traversal and an incomplete-task traversal. State controls which lifecycle actions are valid for an EventTask. Decorator adds organiser approval and urgent handling at runtime without creating many task subclasses. StructureVersion is only the small safety mechanism used to detect hierarchy changes during traversal.

The most important implementation discipline is to preserve the ownership contracts. If the team changes detach/wrap/add semantics, iterator invalidation, or state replacement behaviour, update the UML and this specification immediately so the design, code and demonstration continue to tell the same story.

Appendix A. Fast Interface Freeze Checklist

- Agree exact namespaces (if any) before coding. This specification leaves them optional to reduce friction.
- Agree exact return type for invalid state actions: this design uses bool.
- Agree priority range: recommended 0-10.
- Agree error policy for NULL decorator constructor: recommended std::invalid_argument if exceptions are acceptable; otherwise document/assert.

- Agree whether iterator first() refreshes to the current structure version. This specification recommends yes; after a mutation the safest normal workflow is still to destroy and recreate the iterator.
- Do not change public signatures on one feature branch without notifying the other two people.

Appendix B. Traceability Matrix: Diagram Action -> C++ Operation

Workflow / diagram concept	Traceable C++ operation
Create hierarchy	WorkGroup::addChild
Move/decorate existing task	WorkGroup::detachChild + decorator constructor + WorkGroup::addChild
Mark ready task	ExecutableTask::markReady / EventTask state delegation
Start task	ExecutableTask::start; optionally intercepted by ApprovalRequiredDecorator/UrgentTaskDecorator
Block/resume	EventTask::block / resolveBlock
Review task	submitForReview / approveReview / rejectReview
Full traversal	WorkGroup::createDepthFirstIterator + WorkIterator operations
Incomplete traversal	WorkGroup::createIncompleteTaskIterator + WorkIterator operations
Detect structure change	StructureVersion::bump + iterator isValid/checkVersion
Aggregate cost	WorkGroup::estimatedCost recursively; decorators alter task cost
Aggregate urgency	WorkGroup::priorityScore recursively; UrgentTaskDecorator alters task priority